

Plush Toy ERP 项目完善路线图

当前状态：产品化内测候选

下一目标：可控客户试用候选

长期目标：生产交付候选

策略：当前仍处于开发阶段，允许破坏旧实现，不需要兼容旧 API；优先保证安全、事实源清晰、工程可复现、架构边界正确、客户试用可控。

1. 项目 Review 总结

1.1 当前项目已经具备的基础

当前项目已经不是简单 Demo，已经具备较完整的 ERP 项目骨架：

- 后端已有 Kratos / Ent / Atlas / PostgreSQL 基础。
- 前端已有 Vite / React / Ant Design 业务页面。
- 已有主数据、客户、供应商、联系人、销售订单、采购收货、质检、库存流水、库存余额、库存批次、生产事实、外协事实、出货、库存预留、财务业务事实等模块。
- 已有客户配置、私有化部署模板、导入 dry-run、evidence、QA 脚本和本地 hooks。
- `business_records` 已经基本从正式写入路径退出，开始向 legacy / archive / read-only 方向收敛。
- 文档治理较强，已经有 current source of truth、capability ledger、roadmap、客户导入说明、私有化部署说明等材料。

1.2 当前项目还不能直接客户试用的原因

虽然业务模块已经不少，但还没有达到“客户试用候选”的硬标准，主要阻塞项包括：

1. 源码包仍存在 token / secret 形态内容。
2. 客户导入原始文件名存在编码问题，导致导入测试失败。
3. SQL tracing / logging 存在记录 SQL 参数的风险。
4. 生产配置缺少强制 preflight，placeholder secret/password 可能进入运行环境。
5. 缺少真正 CI，目前主要依赖本地 hooks。
6. 运行时仍然存在 `phase8` 概念。
7. `data` 层仍承担 JSON-RPC dispatch / usecase wiring，分层不干净。
8. 库存预留、出货明细、重复请求等幂等和并发安全需要补强。
9. ERP MVP 仍缺 Product SKU、BOM Version、Purchase Order 等核心内核。
10. 前端大组件、大 CSS、大脚本会影响后续维护速度。

1.3 当前阶段判断

当前项目可以定义为：

产品化内测候选

还不能定义为：

可控客户试用候选
生产交付候选

下一步目标应该是：

先 hardening, 再补 ERP MVP 内核, 再做客户试用包。

2. 总体原则

2.1 允许破坏兼容

当前仍处于开发阶段, 优先保证产品结构正确, 不要为了兼容旧实现而保留错误架构。

可以直接删除、重命名或迁移：

- `phase8` runtime API
- `/erp/phase8/*` 前端路由
- 旧业务 generic write API
- 模糊领域命名
- 造成双真相的旧数据写入路径

不建议保留：

- `phase8.*` alias
- legacy write fallback
- `business_records` 新写入入口
- workflow 直接写库存/出货/财务事实的捷径

2.2 每个业务事实必须只有一个事实源

ERP 最重要的是事实一致性。正式事实源建议如下：

业务事实	唯一事实源
客户	<code>customers</code>
供应商	<code>suppliers</code>
联系人	<code>contacts</code>
产品	<code>products</code>
SKU	<code>product_skus</code>
BOM	<code>bom_headers</code> , <code>bom_items</code>
销售订单	<code>sales_orders</code> , <code>sales_order_items</code>

业务事实	唯一事实源
采购订单	<code>purchase_orders</code> , <code>purchase_order_items</code>
采购收货	<code>purchase_receipts</code>
采购退货	<code>purchase_returns</code>
质检	<code>quality_inspections</code>
库存流水	<code>inventory_txns</code>
库存余额	<code>inventory_balances</code>
库存批次	<code>inventory_lots</code>
库存预留	<code>stock_reservations</code>
生产事实	<code>production_facts</code>
外协事实	<code>outsourcing_facts</code>
出货	<code>shipments</code> , <code>shipment_items</code>
财务业务事实	<code>finance_facts</code>
workflow任务	<code>workflow_tasks</code>
审计	<code>audit_events</code>

禁止出现：

- workflow 直接写库存事实。
- dashboard 自己发明业务状态。
- `business_records` 写正式业务事实。
- 同一业务状态同时由多个表表达。
- 页面状态和后端事实状态不一致。
- 客户导入临时表直接当正式事实源使用。

2.3 Workflow 只做任务流

Workflow 应该负责：

- 任务
- 审批
- 指派
- 协作
- 评论
- 待办
- 状态流转
- 业务对象引用

Workflow 不应该直接负责：

- 库存扣减
- 出货事实
- 财务事实
- 采购收货事实
- 生产事实
- 外协事实
- 正式业务结算

业务事实必须由领域 usecase 写入。

2.4 business_records 只做 legacy / archive

business_records 不再作为正式业务写入口。

允许它存在于：

- 历史只读查询
- archive viewer
- migration bridge
- 老数据兼容查看

禁止它继续承担：

- 新销售订单写入
 - 新采购写入
 - 新库存写入
 - 新出货写入
 - 新财务事实写入
 - 新生产事实写入
 - 新外协事实写入
-

2.5 暂缓 SaaS

当前目标是：

单客户私有化部署
可导入
可备份
可回滚
可审计
可客户试用

暂缓：

- 多租户

- SaaS 计费
 - 租户隔离
 - 在线订阅
 - 公共注册
 - 跨客户模板市场
 - 多客户共享部署
-

3. 优先级定义

3.1 P0：立即阻塞项

P0 必须先完成。没有完成前，不允许进入客户试用。

P0 包括：

1. secret / token 清理与轮换。
 2. 客户导入文件名 / manifest 修复。
 3. SQL tracing 禁止记录参数。
 4. production preflight。
 5. CI baseline。
-

3.2 P1：产品化硬化项

P1 在 P0 后立即做。目标是把项目从“内测候选”推进到“客户试用候选”。

P1 包括：

1. 移除 runtime `phase8`。
 2. JSON-RPC 从 data 层迁出。
 3. 库存预留并发安全。
 4. 出货明细和出货动作幂等。
 5. admin bootstrap 安全。
 6. 生产环境关闭 `auth.register`。
 7. 前端 token 风险缓解。
 8. 安全响应头。
-

3.3 P2：ERP MVP 核心补齐

P2 是客户试用前后的重点。目标是形成真正制造业订单闭环。

P2 包括：

1. Product SKU。
2. BOM Version。
3. Purchase Order。

4. ERP MVP 端到端闭环测试。
5. `business_records` 彻底收敛为 `archive`。
6. 前端大组件拆分。
7. ESLint 收紧。

3.4 P3：客户试用和长期增强

P3 不阻塞第一轮内部 hardening，但进入客户试用前需要逐步完成。

P3 包括：

1. 客户试用 release package。
2. 权限矩阵和审计。
3. 备份恢复演练。
4. 财务事实边界说明。
5. 报表、PDA、扫码、附件等后续能力。

4. 推荐执行顺序

4.1 第一批：P0 安全和可复现

按顺序执行：

1. P0-01 清理 token / secret
2. P0-02 修复客户导入文件名 / source manifest
3. P0-03 禁止 SQL tracing 参数
4. P0-04 production preflight
5. P0-05 CI baseline

完成后目标：

源码包无 secret
导入测试全过
生产危险配置启动失败
CI 可复现
Go / Node / pnpm 版本统一

4.2 第二批：P1 产品化重构

按顺序执行：

1. P1-01 移除 runtime phase8
2. P1-02 JSON-RPC 从 data 层迁出
3. P1-03 库存预留并发安全
4. P1-04 出货明细和出货动作幂等

5. P1-05 admin bootstrap 安全
6. P1-06 生产环境关闭 auth.register
7. P1-07 前端 token 风险缓解和安全响应头

完成后目标：

运行时领域命名正式化
架构分层清晰
关键库存/出货动作并发和幂等安全
认证、注册、bootstrap 风险降低

4.3 第三批：P2 ERP MVP 内核

按顺序执行：

1. P2-01 Product SKU
2. P2-02 BOM Version
3. P2-03 Purchase Order
4. P2-04 ERP MVP 端到端闭环
5. P2-05 business_records 彻底收敛
6. P2-06 前端大组件拆分
7. P2-07 收紧 ESLint

完成后目标：

制造业订单闭环具备正式事实源
核心 ERP 模型更完整
前端维护成本下降
质量规则更严格

4.4 第四批：P3 客户试用准备

按顺序执行：

1. P3-01 客户试用 release package
2. P3-02 权限矩阵和审计
3. P3-03 备份恢复演练
4. P3-04 财务事实边界说明

完成后目标：

可以进入可控客户试用
客户知道能力边界
部署、导入、备份、回滚、验收可执行

5. 任务总表

编号	优先级	任务	是否阻塞客户试用	目标
P0-01	P0	清理 token / secret	是	源码和 release 无敏感信息
P0-02	P0	修复导入文件名 / manifest	是	客户导入测试可复现
P0-03	P0	禁止 SQL 参数 tracing	是	防止 PII/敏感数据进 trace
P0-04	P0	production preflight	是	危险配置启动即失败
P0-05	P0	CI baseline	是	质量门禁不可绕过
P1-01	P1	移除 runtime phase8	是	产品运行时领域命名正式化
P1-02	P1	JSON-RPC 从 data 迁出	是	后端分层清晰
P1-03	P1	库存预留并发安全	是	防止超预留
P1-04	P1	出货幂等	是	防止重复扣库存/回滚
P1-05	P1	admin bootstrap 安全	是	防止自动提权
P1-06	P1	关闭 public register	是	私有化系统默认不可公开注册
P1-07	P1	token 风险和安全头	是	降低 XSS/token 风险
P2-01	P2	Product SKU	是	补核心商品规格模型
P2-02	P2	BOM Version	是	补生产用料版本模型
P2-03	P2	Purchase Order	是	补采购订单闭环
P2-04	P2	MVP e2e 闭环	是	验证真实订单链路
P2-05	P2	business_records 收敛	是	消除双真相
P2-06	P2	前端大组件拆分	否	提升维护性
P2-07	P2	ESLint 收紧	否	提升前端质量
P3-01	P3	客户试用 release package	是	可交付、可回滚、可验收
P3-02	P3	权限矩阵和审计	是	明确角色职责和审计
P3-03	P3	备份恢复演练	是	私有化交付必需
P3-04	P3	财务事实边界	是	避免误导客户

6. P0 任务详情

P0-01：清理源码中的 token / secret

目标

源码仓库和 release zip 中不能包含任何真实 token、password、private key、webhook secret。

当前风险

已发现以下文件存在 token / secret 形态内容或风险：

- web/.npmrc
- web/.yarnrc.yml
- server/configs/prod/config.yaml
- 部分 deployment / evidence / docs 里可能存在 demo password、内网地址、bot token、chatId 等敏感或半敏感内容。

要求

1. 删除源码中的真实 token。

2. 新增 example 文件：

web/.npmrc.example web/.yarnrc.example server/configs/prod/
config.example.yaml .env.example

3. 真实配置文件加入 .gitignore：

.env .env.* !.env.example web/.npmrc web/.yarnrc.yml

4. CI 使用 Secret 注入，不依赖仓库里的真实 token。

5. 更新文档，说明如何配置本地 registry token。

6. 不要在 commit diff 中出现真实 secret。

7. 已泄漏过的 token 必须人工 revoke / rotate。

建议扫描命令

```
grep -RIe "(_authToken|BEGIN PRIVATE KEY|bot[0-9]+:|password:|secret:|  
token:)" .  
--exclude-dir=.git  
--exclude-dir=node_modules  
--exclude-dir=dist  
--exclude-dir=.cache
```

验收标准

- 仓库中没有真实 token。
- .npmrc.example 存在且只包含 \${NPM_TOKEN}。
- .yarnrc.example 存在且只包含 \${NPM_TOKEN}。

- `.env.example` 不含真实密码。
- `scripts/qa/secrets.sh` 能扫描并失败拦截明显 secret。
- CI 中 secret scan 不允许 skip。
- 文档说明真实 secret 只能通过本地环境或 CI Secret 注入。

给 Codex 的提示词

请完成 Task P0-01：清理源码中的 token/secret。

要求：

1. 不要提交任何真实 token、密码、私钥、webhook。
2. 把 `web/.npmrc`、`web/.yarnrc.yml` 改成 example 模板，真实文件加入 `.gitignore`。
3. 检查 `server/configs/prod/config.yaml`，拆成安全的 example 模板。
4. 增强 `scripts/qa/secrets.sh`，使 CI 模式下找不到 `gitleaks/trufflehog` 时直接失败，不允许 skip。
5. 更新 README 或 `docs/deployment` 文档说明如何通过环境变量注入 secret。
6. 运行可用的 secret scan、project scan、node script tests。
7. 输出修改文件、测试结果和仍需人工轮换的 secret 清单。
8. 不要输出 secret 值。

P0-02：修复客户导入原始文件名编码问题

目标

release zip 解压后，客户导入测试必须稳定通过，不能因为中文文件名编码损坏导致失败。

当前问题

源码包内客户原始文件名出现类似：

```
#Uxxxx#Uxxxx...
```

导致导入测试报错：

```
ENOENT: no such file or directory  
expected sourceCount > 5000
```

说明文件虽然包含进来了，但导入链路不可复现。

推荐方案

引入 source manifest，不再依赖中文原始文件名。

新增：

docs/yoyoosun/source-manifest.json

建议字段：

字段	说明
sourceId	稳定 source ID
type	文件类型，如 material_detail / order / customer
originalName	原始文件名，仅用于审计
storedName	仓库或 release 中的稳定文件名
sha256	文件 hash
required	是否必需
notes	备注

示例内容结构：

```
{
  "version": 1,
  "customer": "yoyoosun",
  "sources": [
    {
      "sourceId": "material-detail-26029",
      "type": "material_detail",
      "originalName": "26029#夜樱烬色才料明细表2026-1-19.xlsx",
      "storedName": "26029-material-detail.xlsx",
      "sha256": "",
      "required": true,
      "notes": ""
    }
  ]
}
```

要求

1. 导入工具优先通过 manifest 找文件。
2. 导入工具不再硬编码中文文件名。
3. originalName 仅用于审计和报告。
4. 所有 required source 缺失时，dry-run 必须明确报错。
5. 如果 sha256 存在，必须校验 hash。
6. 文件名编码异常时，不影响导入工具。
7. node --test scripts/**/*.test.mjs 必须全部通过。

验收标准

- 导入测试全部通过。

- release zip 解压后导入 dry-run 能找到所有 required source。
- manifest 中有 `sourceId`、`type`、`storedName`、`originalName`、`sha256`。
- 缺失文件时错误信息清楚。
- 不再因为中文文件名编码导致测试失败。

给 Codex 的提示词

请完成 Task P0-02：修复客户导入原始文件名编码问题。

要求：

1. 增加 docs/yoyoosun/source-manifest.json。
2. 修改 yoyoosun 导入脚本，使其优先通过 manifest 的 storedName/sourceId 找文件，不再硬依赖中文原始文件名。
3. originalName 仅用于审计和报告。
4. 对 required source 做存在性检查。
5. 如果 sha256 字段存在，校验文件 hash。
6. 修复当前失败的 import tests。
7. 运行 `node --test scripts/**/*.test.mjs`。
8. 输出测试结果和导入文件解析策略。

P0-03：禁止 SQL tracing / logging 记录 SQL 参数

目标

生产、预发、客户试用环境中，SQL 参数不得进入 tracing/logging。

当前风险

数据库 tracing 里存在记录 SQL 参数的风险，例如：

```
db.sql.arg.*
```

ERP SQL 参数可能包含：

- 客户联系人
- 手机号
- 地址
- 订单金额
- 财务信息
- token
- 密码 hash
- 客户导入原始字段
- 内部备注

这些不应该进入 trace backend。

要求

1. 移除 SQL args attribute。
2. 生产环境永不记录 SQL 参数。
3. dev 环境默认也不记录。
4. 如确需本地调试，必须显式开启并 redact。
5. Ent debug logger 不能打印敏感 args。
6. 增加测试，确保 production config 下 SQL args tracing 不可开启。

允许记录的 tracing 字段

允许记录：

```
db.system
db.operation
db.table
db.statement.hash
db.duration_ms
db.rows_affected
error.type
```

不允许记录：

```
db.sql.arg.*
db.statement.full_with_values
password
token
secret
phone
address
raw_import_value
```

验收标准

- 代码中不再出现 `db.sql.arg.*`。
- production preflight 会拒绝 SQL args tracing。
- 测试覆盖 tracing redaction。
- 文档说明 production tracing policy。

给 Codex 的提示词

请完成 Task P0-03：禁止 SQL tracing/logging 记录 SQL 参数。

要求：

1. 找到数据库初始化、Ent debug、OpenTelemetry SQL attribute getter 等相关代码。
2. 移除或禁用 SQL 参数写入 trace/log 的逻辑。
3. production/staging/customer-trial 环境下绝不允许记录 SQL args。
4. 如果保留本地 debug 开关，必须默认关闭，并对 password/token/phone/address/raw

```
import fields 做 redact。
```

5. 增加单元测试或配置测试，证明 production 环境不能开启 SQL args tracing。
6. 运行 go test 对应 package；如果 Go 工具链不可用，说明原因。

P0-04：增加 production preflight

目标

生产/客户试用环境启动前必须检查危险配置，发现危险值直接启动失败。

当前问题

配置模板中有 placeholder 值，例如：

```
change-this-prod-postgres-password  
change-this-prod-jwt-secret  
change-this-prod-admin-password
```

placeholder 可以存在于 example 中，但不能进入真实运行环境。

建议新增文件

```
server/internal/conf/preflight.go  
server/internal/conf/preflight_test.go
```

严格环境

以下环境必须执行严格 preflight：

```
prod  
production  
staging  
customer-trial  
private-deploy
```

必须检查的问题

1. JWT secret 是 `change-this`、`default`、`test`、`dev` 或空。
2. DB password 是 `change-this`、`postgres`、`password` 或空。
3. Admin password 是 `12345678`、`admin`、`change-this` 或空。
4. SMS mock enabled。
5. debug seed enabled。
6. debug cleanup enabled。
7. public register enabled。

8. SQL args tracing enabled。
9. CORS allow origin 是 `*`。
10. cookie/session 未启用 secure。
11. Postgres debug enabled。
12. backup path 缺失。
13. private deployment 必要配置缺失。

验收标准

- 危险配置启动失败。
- dev/test 环境可以使用安全降级规则。
- 有完整单元测试覆盖。
- README 或 deploy docs 说明生产配置要求。
- Docker/compose 使用 `.env.example`，不内置真实 secret。

给 Codex 的提示词

请完成 Task P0-04：增加 production preflight。

要求：

1. 新增 `server/internal/conf/preflight.go` 和 `preflight_test.go`。
2. 在 `server` 启动流程中调用 `preflight`。
3. `prod/production/staging/customer-trial/private-deploy` 环境下，发现 placeholder secret、默认密码、SMS mock、debug seed、debug cleanup、public register、SQL args tracing、CORS `*` 等危险配置时直接返回错误并拒绝启动。
4. 更新 `prod config example` 和部署文档。
5. 添加测试覆盖每一种危险配置。
6. 运行 `go test ./internal/conf` 或对应 package。

P0-05：建立真正 CI

目标

项目质量不能只依赖本地 hooks，必须有 GitHub Actions 或等价 CI。

当前问题

项目已有：

```
.githubhooks/pre-commit  
.githubhooks/pre-push  
scripts/qa/*  
scripts/project-scan.sh
```

但缺少真正 CI workflow。

建议新增文件

```
.github/workflows/ci.yml
```

最小 CI 流程

CI 至少要做：

1. checkout
2. setup Node 24.14.0
3. corepack enable
4. pnpm 10.13.1
5. setup Go 1.26.4
6. secret scan
7. `bash scripts/project-scan.sh --strict`
8. `node --test scripts/**/*.test.mjs`
9. `pnpm install --frozen-lockfile`
10. `pnpm lint:check`
11. `pnpm test`
12. `pnpm build`
13. `go test ./...`
14. docker build server
15. docker build web

同时修改

`web/package.json` 增加：

```
"packageManager": "pnpm@10.13.1"
```

拆分 lint：

```
"lint": "pnpm lint:check"
"lint:check": "eslint --max-warnings=0 src"
"lint:fix": "eslint --fix src"
```

统一 Go Dockerfile：

```
ARG GO_BUILDER_IMAGE=golang:1.26.4
```

验收标准

- PR 自动跑 CI。
- CI 失败时不能 merge。
- `lint` 不再默认 `--fix`。
- Node/pnpm/Go 版本统一。

- CI 中 secret scan 不能 skip。
- release zip 可以在干净环境中复现测试。

给 Codex 的提示词

请完成 Task P0-05 : 建立 CI baseline。

要求：

1. 新增 .github/workflows/ci.yml。
2. 使用 Node 24.14.0、pnpm 10.13.1、Go 1.26.4。
3. 运行 project scan、script tests、web install/lint/test/build、go test、docker build。
4. 修改 web/package.json, 增加 packageManager, 并把 lint 拆成 lint:check/lint:fix。
5. 修改 server Dockerfile, 使 Go builder 版本与 go.mod toolchain 保持一致。
6. 如果项目当前测试无法全部通过, 请不要隐藏失败, 修复能修的, 剩余明确列出。

7. P1 任务详情

P1-01：移除 runtime phase8

目标

产品运行时、API、前端路由、代码目录中不再出现 phase8 作为业务概念。

当前问题

代码中仍存在：

```
Phase8Usecase
biz/phase8.go
data/phase8_repo.go
data/jsonrpc_phase8.go
/erp/phase8/facts
phase8.createProductionFact
phase8.createShipment
phase8.createFinanceFact
```

目标命名

直接破坏兼容, 不保留旧 alias。

当前	目标
phase8.createProductionFact	production.createFact
phase8.createOutsourcingFact	outsourcing.createFact
phase8.createShipment	shipment.create
phase8.addShipmentItem	shipment.addItem
phase8.shipShipment	shipment.ship
phase8.cancelShippedShipment	shipment.cancelShipped
phase8.createReservation	inventory.reserve
phase8.cancelReservation	inventory.cancelReservation
phase8.createFinanceFact	finance.createFact

建议文件结构

后端 biz :

```
server/internal/biz/production_fact.go
server/internal/biz/outsourcing_fact.go
server/internal/biz/shipment.go
server/internal/biz/stock_reservation.go
server/internal/biz/finance_fact.go
```

后端 data :

```
server/internal/data/production_fact_repo.go
server/internal/data/outsourcing_fact_repo.go
server/internal/data/shipment_repo.go
server/internal/data/stock_reservation_repo.go
server/internal/data/finance_fact_repo.go
```

前端路由 :

```
/erp/production
/erp/outsourcing
/erp/shipments
/erp/inventory/reservations
/erp/finance/facts
```

验收标准

- runtime API 不再有 `phase8.*`。
- 前端 URL 不再有 `/phase8/`。

- 代码中除 archived docs/evidence 外不再有 `phase8`。
- 测试更新为新 domain。
- 不保留兼容 alias。
- 文档说明 breaking change。

给 Codex 的提示词

请完成 Task P1-01：移除 runtime phase8。

要求：

1. 把后端 JSON-RPC phase8 domain 拆成 production、outsourcing、shipment、inventory、finance。
2. 重命名 Phase8Usecase、phase8 repo、jsonrpc_phase8.go 等运行时代码。
3. 前端路由从 /erp/phase8/facts 改成正式领域页面。
4. 更新测试、菜单、权限、文档。
5. 不要保留 phase8 兼容 alias。
6. archived docs/evidence 中允许保留 phase8 作为历史阶段说明。
7. 运行相关 Go/Web/Node 测试。

P1-02：JSON-RPC 从 data 层迁出

目标

`data` 层只做 repository，不承担 JSON-RPC dispatch、权限、handler、usecase wiring。

当前问题

当前存在类似：

```
server/internal/data/jsonrpc.go
server/internal/data/jsonrpc_business.go
server/internal/data/jsonrpc_phase8.go
```

`data` 层承担了：

- method dispatch
- public method 判断
- auth context
- usecase wiring
- domain route
- error mapping
- business/masterdata/sales/purchase/phase8/debug/admin API 入口

目标结构

```
server/internal/server
server/internal/service/jsonrpc
server/internal/service/jsonrpc/auth
server/internal/service/jsonrpc/admin
server/internal/service/jsonrpc/debug
server/internal/service/jsonrpc/workflow
server/internal/service/jsonrpc/masterdata
server/internal/service/jsonrpc/sales
server/internal/service/jsonrpc/purchase
server/internal/service/jsonrpc/inventory
server/internal/service/jsonrpc/production
server/internal/service/jsonrpc/outourcing
server/internal/service/jsonrpc/shipment
server/internal/service/jsonrpc/finance
server/internal/biz
server/internal/data
```

职责边界

`service/jsonrpc` 负责：

- parse params
- validate request
- extract auth context
- permission check
- call usecase
- map error
- return DTO

`biz` 负责：

- usecase
- business rules
- state machine
- idempotency
- domain event
- transaction boundary intent

`data` 负责：

- Ent query/mutation
- repository
- transaction
- locking
- persistence
- projection

迁移策略

1. 先创建 `internal/service/jsonrpc`。
2. 搬 `auth/admin/debug` 这些低风险 domain。
3. 搬 `masterdata/sales/purchase`。
4. 搬 `production/shipment/inventory/finance`。
5. 删除 `data/jsonrpc*.go`。

验收标准

- `internal/data` 中不再有 JSON-RPC dispatch。
- `data` 包不依赖 `service`。
- `service` 可以依赖 `biz`。
- `biz` 可以依赖 `repo interface`。
- `data` 实现 `repo interface`。
- 所有 JSON-RPC 测试通过。

给 Codex 的提示词

请完成 Task P1-02 的第一阶段：把 JSON-RPC dispatch 从 `data` 层迁到 `internal/service/jsonrpc`。

要求：

1. 先不要大范围重写业务逻辑。
2. 创建 `internal/service/jsonrpc` 包。
3. 把 `method dispatch`、`public method`、`error mapping` 从 `data/jsonrpc.go` 迁出。
4. `data` 层只保留 `repository` 和 `persistence`。
5. 保持现有 API 行为，除已经明确要求破坏兼容的 `phase8` 重命名外。
6. 增加或更新测试。
7. 完成后列出仍留在 `data` 层的非 `repository` 逻辑，作为下一步清理清单。

P1-03：库存预留并发安全

目标

并发创建库存预留时，不能超过可用库存。

风险场景

库存 `available = 100`

请求 A 同时进入，检查可用 100，预留 80

请求 B 同时进入，检查可用 100，预留 80

最终 `active reservation = 160`

要求

1. 创建 reservation 必须在事务内执行。
2. 对对应 `inventory_balance` 行加锁。
3. 使用 `SELECT ... FOR UPDATE` 或数据库 advisory lock。
4. 并发请求不能超过可用库存。
5. reservation 支持 idempotency key。
6. cancel reservation 恢复可用库存。
7. ship shipment 时正确消耗 reservation。

测试场景

必须新增：

1. concurrent reservations cannot exceed available stock
2. repeated reservation idempotency key returns same result
3. reservation cancel restores availability
4. reservation cannot be canceled twice
5. shipment consumes active reservation
6. shipment fails if reservation insufficient or invalid

验收标准

- 并发测试稳定通过。
- 事务中锁定库存余额。
- 失败请求有明确错误码。
- 重复请求不重复创建 reservation。
- 取消操作幂等或明确拒绝重复取消。

给 Codex 的提示词

请完成 Task P1-03：库存预留并发安全。

要求：

1. 找到库存预留创建逻辑。
2. 在事务内锁定 `inventory_balance` 对应行，避免并发超预留。
3. 增加 idempotency key 支持。
4. 增加并发测试：多个 goroutine 同时预留，总预留不能超过可用库存。
5. 增加取消预留、重复取消、重复创建的测试。
6. 不要让 workflow 直接修改库存事实。
7. 运行相关 Go 测试。

P1-04：出货明细和出货动作幂等

目标

重复添加出货明细、重复发货、重复取消发货不能造成重复库存扣减或重复回滚。

要求

1. `shipment_items` 增加业务唯一约束。
2. `shipment.addItem` 支持 idempotency。
3. `shipment.ship` 重复调用不能重复扣库存。
4. `shipment.cancelShipped` 重复调用不能重复回滚库存。
5. 所有库存流水必须有：

`source_type source_id source_line_id idempotency_key`

推荐唯一键

二选一：

`(shipment_id, sales_order_item_id)`

或：

`(shipment_id, source_type, source_id, source_line_id)`

测试场景

1. repeated add item does not create duplicate line
2. repeated ship does not double deduct inventory
3. repeated cancel shipped does not double reverse inventory
4. cannot add item after shipped
5. cannot ship empty shipment
6. cannot cancel unshipped shipment as shipped cancellation

验收标准

- 数据库层有唯一约束。
- usecase 层有幂等保护。
- 库存流水不会重复生成。
- 测试覆盖重复请求。

给 Codex 的提示词

请完成 Task P1-04：出货明细和出货动作幂等。

要求：

1. 为 `shipment_items` 增加合适的唯一约束。
2. 修改 `add item/ship/cancel shipped` 逻辑，避免重复明细、重复扣库存、重复回滚。
3. 所有库存流水关联 `source_type/source_id/source_line_id/idempotency_key`。
4. 增加 migration。

5. 增加重复请求测试。
6. 运行 Go 测试和 migration lint。

P1-05：admin bootstrap 安全

目标

生产环境中，系统不能因为配置错误把已有普通用户自动提升为 super-admin。

当前风险

如果 admin username 已存在但不是 super-admin，初始化逻辑可能自动提权。

要求

1. 只允许首次 bootstrap 创建 super admin。
2. 已存在用户不自动提权。
3. 提权必须通过显式命令或管理操作。
4. 生产环境需要显式：

`BOOTSTRAP_ADMIN_ONCE=true`

5. bootstrap 成功后写入 marker，不再重复执行。
6. 所有 bootstrap 行为写入 audit event。

验收标准

- 已存在普通用户不会自动变 super-admin。
- bootstrap 只能执行一次。
- 重复 bootstrap 失败或 no-op。
- production 环境没有显式 flag 时不执行 bootstrap。
- 测试覆盖。

给 Codex 的提示词

请完成 Task P1-05：admin bootstrap 安全。

要求：

1. 找到初始化 admin 用户的逻辑。
2. 改成首次 bootstrap-only，不允许自动提升已有用户权限。
3. 生产环境必须显式配置 `BOOTSTRAP_ADMIN_ONCE=true` 才允许创建初始 super admin。
4. bootstrap 完成后写 marker。
5. 写 audit event。
6. 增加测试覆盖已有普通用户、重复 bootstrap、未开启 flag 的生产环境。

P1-06：生产环境关闭 `auth.register`

目标

ERP 私有化系统默认不允许公开注册。

要求

1. `auth.register` 在 production/staging/customer-trial 默认关闭。
2. dev/test 可开启。
3. 如确需注册，必须配置：

`AUTH_PUBLIC_REGISTER_ENABLED=true`

4. 即使开启，也需要：

rate limit 最小角色 审计日志 邀请或审批机制

验收标准

- production preflight 会拒绝 public register。
- JSON-RPC public methods 中 register 受配置控制。
- 测试覆盖 production disabled。
- 文档说明默认关闭。

给 Codex 的提示词

请完成 Task P1-06：生产环境关闭 `auth.register`。

要求：

1. 找到 JSON-RPC public method 列表。
2. `auth.register` 在 production/staging/customer-trial 中默认关闭。
3. dev/test 可以开启。
4. 增加配置项 `AUTH_PUBLIC_REGISTER_ENABLED`。
5. production preflight 中如果开启 public register 则失败，除非明确允许试验模式。
6. 增加测试。

P1-07：前端 token 风险缓解和安全响应头

目标

降低 access token 存在 localStorage 带来的 XSS 风险，并补齐基础安全响应头。

短期要求

1. access token 有较短有效期。

2. admin token 比普通 token 更短。
3. 前端避免在 console 打印 token。
4. API error 不返回 token。
5. 增加 CSP。
6. 增加安全响应头。
7. localStorage key 命名清晰，支持 logout 清理全部 token。
8. XSS 风险文档化。

建议安全响应头

```
Content-Security-Policy
X-Content-Type-Options: nosniff
Referrer-Policy
Permissions-Policy
X-Frame-Options 或 frame-ancestors
Strict-Transport-Security
```

中期目标

迁移到：

```
HttpOnly Secure SameSite cookie
refresh token rotation
jti
revocation list
audience 分离
```

验收标准

- localStorage 不打印 token。
- logout 清理所有 auth storage。
- 静态服务有安全头。
- 文档记录中期迁移计划。

给 Codex 的提示词

请完成 Task P1-07：前端 token 风险缓解。

要求：

1. 搜索前端是否有 console 打印 token、用户敏感信息、Authorization header。
2. 修复 logout，确保清理所有 auth storage。
3. 增加或配置安全响应头：CSP、X-Content-Type-Options、Referrer-Policy、Permissions-Policy、frame-ancestors/X-Frame-Options。
4. 检查 access token/admin token TTL 配置。
5. 增加文档：当前 localStorage 风险和后续 HttpOnly cookie 迁移计划。
6. 不要在代码或测试中写真实 token。

8. P2 任务详情

P2-01：Product SKU

目标

建立稳定 SKU 模型，支撑销售、BOM、库存、出货、成本。

新增实体

product_skus

建议字段

字段	说明
id	主键
sku_code	SKU 编码
product_id	所属产品
name	SKU 名称
spec	规格
unit_id	默认单位
barcode	条码
customer_sku_code	客户 SKU
supplier_sku_code	供应商 SKU
status	状态
attributes_json	扩展属性
created_at	创建时间
updated_at	更新时间
deleted_at	软删除

状态

ACTIVE
INACTIVE
ARCHIVED

要求

1. SKU code 唯一。
2. SKU 关联 product。
3. 销售订单明细可引用 SKU。
4. BOM 可引用 SKU。
5. 库存批次/余额可以引用 SKU。
6. 前端提供最小 CRUD。
7. 导入工具可以映射 SKU。
8. 不允许通过 `business_records` 写入 SKU。

验收标准

- Ent schema + migration。
- repository + usecase。
- JSON-RPC API。
- 权限控制。
- 测试。
- 前端最小页面。
- 文档更新。

给 Codex 的提示词

请完成 Task P2-01：新增 Product SKU 模型。

要求：

1. 新增 Ent schema `product_skus` 和 migration。
2. 实现 repository/usecase/API。
3. SKU code 唯一，支持 status。
4. `sales_order_items`、`bom_headers` 或 `bom_items` 按需要支持 `sku_id`。
5. 前端增加 SKU 管理最小页面。
6. 增加测试和文档。
7. 不要复用 `business_records` 写入 SKU。

P2-02：BOM Version

目标

建立版本化 BOM，支撑玩具生产用料、损耗率、替代料和量产版本管理。

新增或完善实体

```
bom_headers
bom_items
```

bom_headers 字段建议

字段	说明
id	主键
bom_code	BOM 编码
product_id	产品
sku_id	SKU
version	BOM 版本
status	状态
effective_from	生效开始
effective_to	生效结束
notes	备注
created_at	创建时间
updated_at	更新时间
deleted_at	软删除

bom_items 字段建议

字段	说明
id	主键
bom_id	BOM header
material_id	物料
material_sku_id	物料 SKU
qty	用量
unit_id	单位
loss_rate	损耗率
substitute_group	替代料组
required_flag	是否必需
sort_order	排序
notes	备注
created_at	创建时间
updated_at	更新时间
deleted_at	软删除

状态

DRAFT
ACTIVE
ARCHIVED

规则

1. 同一个 product/sku 同一时间只能有一个 ACTIVE BOM。
2. ACTIVE BOM 不允许直接修改明细，必须复制新版本。
3. DRAFT 可以修改。
4. ARCHIVED 只读。
5. BOM item qty 必须大于 0。
6. loss_rate 必须做合理范围校验。
7. BOM 激活必须写 audit event。

验收标准

- 有 version。
- 有状态机。
- 有 active 唯一规则。
- 有 copy new version。
- 有最小 UI。
- 有测试。

给 Codex 的提示词

请完成 Task P2-02 : BOM Version。

要求：

1. 完善 bom_headers 和 bom_items。
2. 支持 DRAFT/ACTIVE/ARCHIVED 状态。
3. 同一个 product/sku 同一时间只能有一个 ACTIVE BOM。
4. ACTIVE BOM 不允许直接修改，必须 copy new version。
5. BOM item 支持 qty、unit、loss_rate、substitute_group、required_flag。
6. 增加 usecase/API/前端最小页面。
7. 增加状态机和唯一性测试。

P2-03 : Purchase Order

目标

补齐采购订单，避免长期只有采购收货而没有采购订单。

新增实体

purchase_orders
purchase_order_items

purchase_orders 字段建议

字段	说明
id	主键
po_no	采购单号
supplier_id	供应商
status	状态
order_date	下单日期
expected_arrival_date	预计到货日期
currency	币种
total_amount	总金额
notes	备注
created_by	创建人
created_at	创建时间
updated_at	更新时间
deleted_at	软删除

purchase_order_items 字段建议

字段	说明
id	主键
purchase_order_id	采购订单
material_id	物料
sku_id	SKU
qty	采购数量
received_qty	已收数量
unit_id	单位
unit_price	单价
amount	金额

字段	说明
<code>expected_arrival_date</code>	预计到货日期
<code>notes</code>	备注
<code>created_at</code>	创建时间
<code>updated_at</code>	更新时间
<code>deleted_at</code>	软删除

状态

DRAFT
 APPROVED
 PARTIALLY_RECEIVED
 RECEIVED
 CANCELLED
 CLOSED

规则

1. DRAFT 可编辑。
2. APPROVED 后可收货。
3. 收货引用 PO item。
4. 收货数量不能超过订单数量，除非显式允许 over-receive。
5. 退货引用 receipt 或 PO item。
6. CLOSED 后只读。
7. 状态变化必须写 audit event。

验收标准

- 有 PO/PO item schema。
- purchase_receipts 引用 PO item。
- 状态自动根据收货数量更新。
- 有最小 UI。
- 有测试。

给 Codex 的提示词

请完成 Task P2-03 : Purchase Order。

要求：

1. 新增 purchase_orders 和 purchase_order_items。
2. 实现 DRAFT/APPROVED/PARTIALLY_RECEIVED/RECEIVED/CANCELLED/CLOSED 状态。
3. purchase_receipts 支持引用 purchase_order_item。
4. 收货时校验数量，更新 received_qty 和 PO 状态。
5. 增加 repository/usecase/API/前端最小页面。
6. 增加测试：超收、部分收货、完全收货、取消、关闭。

P2-04：ERP MVP 端到端闭环

目标

跑通一条真实制造业订单链路。

链路

客户
销售订单
销售订单明细
SKU
BOM
采购订单
采购收货
质检
入库
生产事实
外协事实
出货
财务业务事实

要求

1. 建立一个 e2e seed。
2. 建立一个 e2e test。
3. 每一步有明确事实源。
4. 不依赖 `business_records` 写入。
5. 不依赖 `phase8` API。
6. 每一步有 audit event。
7. 失败时能清楚定位在哪一步。

建议新增脚本

```
scripts/e2e/manufacturing-mvp.mjs
```

建议输出 report

```
{
  "customerCreated": true,
  "salesOrderCreated": true,
  "skuLinked": true,
  "bomActivated": true,
  "purchaseOrderApproved": true,
  "receiptCreated": true,
  "qualityInspectionPassed": true,
```

```
"inventoryIncreased": true,
"productionFactCreated": true,
"shipmentShipped": true,
"financeFactCreated": true
}
```

验收标准

- e2e test 通过。
- CI 或 qa full 能运行该测试。
- 失败时能输出失败步骤。
- 不走旧 `business_records` 写入。
- 不走 `phase8` API。

给 Codex 的提示词

请完成 Task P2-04 : ERP MVP 端到端闭环测试。

要求：

1. 编写一个 e2e seed/test, 从客户销售订单开始, 到 SKU/BOM/采购/收货/质检/入库/生产/出货/财务事实结束。
2. 不允许使用 `business_records` 写入正式事实。
3. 不允许使用 `phase8` API。
4. 输出 JSON report。
5. 将该测试加入 CI 或 qa full。

P2-05：彻底收敛 `business_records`

目标

`business_records` 只作为 legacy/archive/read-only, 不再出现在正式业务写入路径。

要求

1. 标记所有 `business_records` 写入入口。
2. 删除或禁用写 API。
3. 前端旧业务页面改成 archive viewer 或迁移到正式领域页面。
4. Dashboard 不再读取 `business_records` 作为正式事实。
5. 文档说明 `business_records` 的历史定位。
6. 如果仍保留表, 菜单和文档要体现 archive/legacy。

验收标准

- 代码中无正式业务写入 `business_records`。
- 前端不再把 `business_records` 作为主业务入口。
- 测试确保 Create/Update/Delete business record 失败或只允许 archive migration。
- 文档更新。

给 Codex 的提示词

请完成 Task P2-05：彻底收敛 business_records。

要求：

1. 搜索所有 business_records 写入路径。
2. 确保正式业务不能再创建/更新/删除 business_records。
3. 前端对应入口改成 Legacy Archive, 只读。
4. dashboard 不从 business_records 生成正式 KPI。
5. 增加测试, 确保 legacy 写入被拒绝。
6. 更新文档。

P2-06：拆分前端大组件

目标

降低前端维护成本，避免一个页面文件持续膨胀。

当前大文件

```
web/src/erp/pages/BusinessModulePage.jsx
web/src/erp/mobile/pages/MobileRoleTasksPage.jsx
web/src/erp/styles/app.css
web/scripts/styleL1.mjs
```

BusinessModulePage 拆分建议

```
web/src/erp/pages/business-module/
  index.jsx
  BusinessModuleShell.jsx
  BusinessModuleFilterBar.jsx
  BusinessModuleTable.jsx
  BusinessModuleToolbar.jsx
  BusinessModuleDetailDrawer.jsx
  BusinessModuleEmptyState.jsx
  hooks/
    useBusinessModuleQuery.js
    useBusinessModuleActions.js
    useBusinessModuleSelection.js
  definitions/
    salesOrder.js
    purchaseReceipt.js
    inventory.js
    shipment.js
```

MobileRoleTasksPage 拆分建议

```
web/src/erp/mobile/pages/mobile-role-tasks/  
  index.jsx  
  MobileTaskList.jsx  
  MobileTaskDetail.jsx  
  MobileTaskActionPanel.jsx  
  MobileTaskEvidencePanel.jsx  
  RoleTaskTabs.jsx  
  hooks/  
    useMobileTasks.js  
    useTaskAction.js  
    useRoleTaskPermissions.js
```

CSS 拆分建议

```
web/src/erp/styles/  
  tokens.css  
  layout.css  
  erp-shell.css  
  business-module.css  
  mobile-tasks.css  
  print.css
```

执行策略

1. 先抽 hooks。
2. 再抽 components。
3. 再抽 definitions。
4. 最后拆 CSS。
5. 每一步保持 UI 行为不变。
6. 不要一次重写所有页面。

验收标准

- 单文件行数明显下降。
- UI 行为不变。
- Web tests 通过。
- lint 通过。
- 没有引入循环依赖。

给 Codex 的提示词

请完成 Task P2-06 的第一阶段：拆分 BusinessModulePage。

要求：

1. 不改变 UI 行为。
2. 先抽 hooks：query、actions、selection。

3. 再抽 filter/table/toolbar/detail drawer。
4. 保持 import 清晰。
5. 不要顺手重写业务逻辑。
6. 运行 web lint/test/build。
7. 输出拆分后的文件结构和后续还需拆分的部分。

P2-07：收紧 ESLint

目标

提高前端质量门槛，避免 hooks 和 unused vars 问题长期积累。

当前问题

部分规则过松，例如：

```
no-unused-vars
react-hooks/exhaustive-deps
```

`lint` 默认 auto-fix 不适合作为 CI gate。

要求

1. `lint` 改为 check。
2. `lint:fix` 才允许 auto-fix。
3. `react-hooks/exhaustive-deps` 至少 warn，最终 error。
4. `no-unused-vars` 至少 warn，最终 error。
5. CI 使用 `lint:check`。
6. 对老代码可以分目录逐步启用。

验收标准

- package scripts 清晰。
- CI 不会自动修改代码。
- lint 规则逐步收紧。
- 文档说明迁移计划。

给 Codex 的提示词

请完成 Task P2-07：收紧 ESLint。

要求：

1. 修改 web/package.json, 把 lint 拆成 lint:check 和 lint:fix。
2. CI 使用 lint:check。
3. react-hooks/exhaustive-deps 从 off 改成 warn 或 error。
4. no-unused-vars 从 off 改成 warn 或 error。

5. 修复因此暴露的低风险问题。
6. 对暂时无法修复的大文件，使用局部注释并写 TODO，不要全局关闭规则。

9. P3 任务详情

P3-01：客户试用 release package

目标

生成一个可交付、可审计、可回滚的客户试用包。

推荐结构

```
release/  
  source.tar.gz  
  docker-compose.yml  
  .env.example  
  migration-status.txt  
  seed-report.json  
  import-dry-run-report.json  
  unresolved-import-queue.json  
  smoke-test-report.json  
  security-scan-report.json  
  backup-restore-report.json  
  acceptance-checklist.md  
  known-limitations.md
```

known-limitations.md 必须说明

- 正式能力
- 试用能力
- 模拟能力
- 文档能力
- 不承诺能力
- 已知风险
- 客户验收步骤
- 回滚步骤

验收标准

- release package 不含 secret。
- import dry-run 可复现。
- backup/restore 有报告。
- migration status 有报告。
- smoke test 有报告。

- 客户知道哪些能力不承诺。

给 Codex 的提示词

请完成 Task P3-01：客户试用 release package。

要求：

1. 增加 scripts/release/build-customer-trial-package.mjs。
2. 生成 release/ 目录, 包含 source、compose、env example、migration status、seed report、import dry-run report、smoke test report、security scan report、backup restore report、acceptance checklist、known limitations。
3. release 包构建前必须跑 secret scan。
4. 不允许把真实 token、密码、客户 raw credential 打进 release。
5. 文档说明如何验收和回滚。

P3-02：权限矩阵和审计

目标

客户试用前明确每个角色能做什么，关键操作都有审计。

角色建议

```
super_admin
admin
sales
purchase
warehouse
quality
production
outsourcing
finance
viewer
```

关键操作审计

必须审计：

- login
- logout
- create/update/delete master data
- approve purchase order
- receive goods
- quality pass/reject
- inventory adjustment
- create/cancel reservation
- ship shipment

- cancel shipped shipment
- create finance fact
- bootstrap admin
- change permissions
- run import
- run migration

验收标准

- 有权限矩阵文档。
- 关键 usecase 写 audit event。
- 前端菜单按权限显示。
- 后端 API 强制权限，不只依赖前端隐藏。

给 Codex 的提示词

请完成 Task P3-02：权限矩阵和审计。

要求：

1. 增加 docs/security/permission-matrix.md。
2. 列出角色、菜单、API、动作权限。
3. 检查关键 usecase 是否写 audit event。
4. 补齐缺失的 audit event。
5. 确保后端强制权限，前端隐藏菜单不能作为唯一权限控制。
6. 增加权限测试。

P3-03：备份恢复演练

目标

私有化部署必须能备份、恢复、验证。

要求

1. 增加备份脚本。
2. 增加恢复脚本。
3. 增加 restore verification。
4. 生成 report。
5. customer trial release 包必须包含 backup restore report。

建议文件

```
scripts/deploy/backup.sh
scripts/deploy/restore.sh
scripts/deploy/verify-restore.sh
docs/deployment/backup-restore.md
```


验收标准

- 可以从备份恢复数据库。
- 恢复后 smoke test 通过。
- 备份文件不包含在源码仓库。
- 文档说明 RPO/RTO。
- release package 包含备份恢复报告。

给 Codex 的提示词

请完成 Task P3-03：备份恢复演练。

要求：

1. 增加 scripts/deploy/backup.sh 和 restore.sh，或检查现有脚本并完善。
2. 增加 restore verification 脚本。
3. 输出 backup-restore-report.json。
4. 文档说明私有化部署如何备份、恢复、验证。
5. 不要把真实备份文件提交到仓库。

P3-04：财务事实边界说明

目标

明确当前 `finance_facts` 是业务财务事实，不是正式总账系统。

当前定位

`finance_facts` 可以表达：

- 应收
- 应付
- 收款
- 付款
- 结算状态
- 业务金额
- 关联订单/出货/采购

但不等于：

- 总账
- 会计凭证
- 税务发票
- 期末结账
- 法定财务报表

未来正式财务账可能需要

```
accounts
journal_vouchers
ledger_entries
period_close
reconciliation
tax_invoices
```

要求

1. 文档明确财务能力边界。
2. UI 文案避免误导客户。
3. known limitations 说明该限制。
4. capability ledger 中标记财务能力成熟度。

验收标准

- docs/product/finance-boundary.md 存在。
- UI 文案不声称完整财务系统。
- known-limitations.md 说明该限制。
- capability ledger 更新。

给 Codex 的提示词

请完成 Task P3-04：财务事实边界说明。

要求：

1. 增加 docs/product/finance-boundary.md。
2. 明确 finance_facts 是业务财务事实，不是正式总账/税务/凭证系统。
3. 检查前端文案，避免误导客户。
4. 更新 known-limitations.md 和 capability ledger。

10. 每个阶段的完成定义

10.1 P0 完成定义

P0 完成必须满足：

- [] 源码中无真实 token/secret。
- [] 泄漏过的 token 已人工轮换。
- [] .npmrc / .yarnrc.yml 真实文件不入库。
- [] import tests 全部通过。
- [] SQL tracing 不记录参数。
- [] production preflight 拒绝危险配置。
- [] CI 存在并跑核心检查。

- [] Node/pnpm/Go 版本统一。
 - [] release zip 可在干净环境中进行基础验证。
-

10.2 P1 完成定义

P1 完成必须满足：

- [] runtime API 不再有 `phase8`。
 - [] 前端路由不再有 `/phase8`。
 - [] data 层不再做 JSON-RPC dispatch。
 - [] 库存预留并发不会超卖/超预留。
 - [] 出货重复请求不会重复扣库存。
 - [] 出货取消重复请求不会重复回滚库存。
 - [] admin bootstrap 不会自动提权已有用户。
 - [] production 默认关闭 public register。
 - [] 前端 auth storage 有风险缓解。
 - [] 安全响应头存在。
 - [] 权限控制后端强制执行。
-

10.3 P2 完成定义

P2 完成必须满足：

- [] Product SKU 上线。
 - [] BOM Version 上线。
 - [] Purchase Order 上线。
 - [] 采购收货引用 PO item。
 - [] ERP MVP e2e 测试通过。
 - [] `business_records` 仅 archive/read-only。
 - [] 前端关键大组件已拆分。
 - [] ESLint 规则收紧。
 - [] 领域事实源清晰。
 - [] 不存在 workflow 直接写业务事实的新增路径。
-

10.4 P3 完成定义

P3 完成必须满足：

- [] customer trial release package 可生成。
- [] release package 不含 secret。
- [] migration status report 存在。
- [] import dry-run report 存在。
- [] smoke test report 存在。
- [] backup restore report 存在。
- [] acceptance checklist 存在。

- [] known limitations 存在。
- [] 权限矩阵存在。
- [] 关键操作有 audit event。
- [] 客户试用边界明确。

11. 不建议现在做的事

在 P0/P1/P2 未完成前，不建议做：

1. SaaS 多租户。
2. 复杂 BI 报表。
3. AI 自动排产。
4. PDA / 扫码 / 拍照 / 附件。
5. 复杂财务总账。
6. 在线计费。
7. 客户模板市场。
8. 多客户共享部署。
9. 大规模 UI 重设计。
10. 兼容旧 `phase8` API。
11. 复杂插件系统。
12. 大规模权限 DSL。
13. 引入微服务拆分。

原因：

这些会扩大复杂度，但不能解决当前最关键的安全、事实源、架构和交付问题。

12. Codex 通用提示词模板

后续每次叫 Codex，可以用这个模板。

你现在在 `plush-toy-erp` 项目中工作。

请只完成 `roadmap` 中的任务：<任务编号和名称>。

重要约束：

1. 当前项目仍处于开发阶段，可以破坏旧实现，不需要兼容旧 API，除非任务明确要求。
2. 不允许提交任何真实 token、密码、私钥、webhook、客户凭证。
3. 不允许让 `business_records` 重新成为正式业务事实写入源。
4. 不允许让 `workflow` 直接写库存、出货、采购、财务事实。
5. 不允许新增 `runtime phase8` API 或 `phase8` 前端路由。
6. 每个业务事实必须有明确唯一事实源。
7. `data` 层只做 `repository/persistence`，不要新增 `JSON-RPC dispatch`。
8. 修改后必须运行相关测试；如果测试无法运行，要说明原因。

9. 输出修改文件列表、测试结果、风险、未完成项。

请按以下步骤执行：

1. 阅读相关代码和文档。
2. 给出简短实现计划。
3. 修改代码。
4. 增加或更新测试。
5. 运行测试。
6. 总结结果。

任务详情：

<粘贴对应 Task 的内容>

13. Codex 每次完成后的验收输出格式

Codex 每次完成任务后，必须输出：

完成摘要

- 完成了什么：
- 没有做什么：
- 是否破坏兼容：
- 是否涉及 migration：
- 是否涉及安全：

修改文件

- path/to/file1
- path/to/file2

测试结果

- 命令：
- 结果：
- 未运行测试：
- 未运行原因：

风险

- 风险 1：
- 风险 2：

后续建议

- 建议 1：
- 建议 2：

14. 人工 Review 清单

每个 Codex PR 完成后，人工 Review 时检查：

- ☐ 是否改了无关大范围文件。
- ☐ 是否引入真实 secret。
- ☐ 是否绕过测试。
- ☐ 是否只改前端没改后端权限。
- ☐ 是否只改文档没改代码。
- ☐ 是否只改 usecase 没改 migration。
- ☐ 是否引入新的双真相。
- ☐ 是否让 workflow 写业务事实。
- ☐ 是否让 `business_records` 重新承担正式写入。
- ☐ 是否保留 `phase8` alias。
- ☐ 是否缺少 idempotency。
- ☐ 是否缺少并发测试。
- ☐ 是否缺少 audit event。
- ☐ 是否破坏导入 dry-run。
- ☐ 是否破坏 private deployment。
- ☐ 是否破坏 existing seed。
- ☐ 是否更新 known limitations。
- ☐ 是否更新 capability ledger。
- ☐ 是否增加必要 migration。
- ☐ 是否更新前端菜单和权限。
- ☐ 是否确保后端权限强制执行。
- ☐ 是否能在干净环境复现。

15. 项目阶段目标

15.1 当前阶段

产品化内测候选

说明：

- 内部模拟闭环较完整。
 - 领域事实模型正在形成。
 - 客户导入和私有化材料较完整。
 - 但安全、CI、preflight、runtime 命名、架构分层仍未达客户试用标准。
-

15.2 下一阶段目标

可控客户试用候选

进入条件：

- P0 全部完成。
- P1 核心完成。
- P2 至少完成 SKU / BOM / Purchase Order / MVP e2e。
- 客户 release package 可生成。
- known limitations 清楚。
- backup restore 有演练报告。
- 权限矩阵和审计初步完成。

15.3 生产交付候选条件

生产交付至少还需要：

1. 客户试用通过。
2. 真实导入闭环通过。
3. 备份恢复演练通过。
4. 权限矩阵验收。
5. 审计日志覆盖关键操作。
6. 安全扫描通过。
7. 性能 smoke test 通过。
8. migration rollback 策略明确。
9. release package 可重复构建。
10. 客户签收功能边界。
11. 运维手册齐全。
12. 故障恢复流程演练通过。

16. 最终路线总结

最重要的排序：

1. 安全和可复现。
2. runtime 产品化命名和后端分层。
3. 库存/出货/财务事实并发与幂等。
4. SKU / BOM / Purchase Order。
5. 客户试用 release package。
6. 前端拆分和质量规则。
7. 报表、扫码、PDA、SaaS、多租户。

一句话原则：

先把项目变成安全、可复现、事实源清晰的 ERP 内核；
再扩展客户试用、报表、移动端和 SaaS。

17. 最短执行版本

如果只想先推进最关键的 10 个 Codex goal，建议按这个顺序：

1. P0-01：清理 token / secret。
2. P0-02：修复导入 manifest。
3. P0-03：禁止 SQL 参数 tracing。
4. P0-04：production preflight。
5. P0-05：CI baseline。
6. P1-01：移除 runtime phase8。
7. P1-02：JSON-RPC 从 data 层迁出。
8. P1-03：库存预留并发安全。
9. P1-04：出货幂等。
10. P2-01：Product SKU。

完成这 10 个后，项目质量会明显提升，然后再进入 BOM、Purchase Order、MVP e2e 和客户试用包。